

# カスタマイズされた PowerShell による ファイルレス攻撃のアーティファクト保存手法

木村 将人<sup>†</sup> 劉 クリス<sup>††</sup> 上原哲太郎<sup>†††</sup>

<sup>†</sup> 立命館大学大学院情報理工学研究科 〒525-8577 滋賀県草津市野路東 1-1-1  
<sup>††</sup> デロイト トーマツ サイバー合同会社 〒100-0005 東京都千代田区丸の内 3-2-3  
<sup>†††</sup> 立命館大学情報理工学部 〒525-8577 滋賀県草津市野路東 1-1-1  
E-mail: <sup>†</sup>kimura@cysec.cs.ritsumeai.ac.jp

**あらまし** 多くの攻撃者は、標的マシンにファイルを生成せずに攻撃を行うファイルレス攻撃において、PowerShell を利用する。PowerShell を用いたファイルレス攻撃によるインシデントが発生した場合、その調査手法として主にログ解析が用いられる。PowerShell の操作に関わるログは、PowerShell に標準に備わるログシステムにより、Windows イベントログの 1 つとして記録される。ログ解析を確実に行うためには、PowerShell ログシステムによって記録されるログを保護する必要がある。一方で、多くの攻撃者は、攻撃に関わるアーティファクトを標的マシンに極力残さないアンチ・フォレンジック手法を攻撃に組み込む。そのため、既存の PowerShell ログシステムにおいて、ログ解析に必要なログを確実に保全できない可能性がある。そこで、本研究では PowerShell を用いたファイルレス攻撃を想定し、PowerShell 内部に独自ログシステムを組み込み、直接ローカル外ログサーバにログを保全する手法を提案する。

**キーワード** デジタル・フォレンジック, ファイルレス攻撃, PowerShell, ログ保全

## Preservation Techniques of Fileless Attack Artifacts using Customized PowerShell

Masahito KIMURA<sup>†</sup>, Chris LIU<sup>††</sup>, and Tetsutaro UEHARA<sup>†††</sup>

<sup>†</sup> Graduate School of Information Science and Engineering, Ritsumeikan University  
1-1-1, Noji-higashi, Kusatsu-shi, Shiga, 525-8577 Japan  
<sup>††</sup> Deloitte Tohmatsu Cyber LLC  
Marunouchi Nijubashi Building 3-2-3 Marunouchi, Chiyoda-ku (Reception 17F) Tokyo  
<sup>†††</sup> Graduate School of Information Science and Engineering, Ritsumeikan University  
1-1-1, Noji-higashi, Kusatsu-shi, Shiga, 525-8577 Japan  
E-mail: <sup>†</sup>kimura@cysec.cs.ritsumeai.ac.jp

**Abstract** Many attackers use PowerShell in fileless attacks that do not generate files on the target machine. When a fileless attack incident using PowerShell occurs, log analysis is the main method used to investigate the incident. Logs related to PowerShell operations are recorded as one of the Windows event logs by the log system that comes standard with PowerShell. In order to ensure log analysis, it is necessary to protect the logs recorded by the PowerShell logging system. On the other hand, many attackers incorporate anti-forensic techniques into their attacks that leave as few artifacts as possible on the target machine. Therefore, existing PowerShell logging systems may not be able to reliably preserve logs necessary for log analysis. Therefore, we propose a method to preserve logs directly to an off-local log server by building an original log system inside PowerShell for file-less attacks using PowerShell.

**Key words** Digital Forensics, Fileless Attacks, PowerShell, Log Preservation

## 1. はじめに

近年の標的型サイバー攻撃では、標的マシン上で悪意のあるファイルを用いて攻撃を行うファイルベース攻撃が主流である。一方で、標的マシンにファイルを生成せずに攻撃を行うファイルレス攻撃の登場により、サイバー攻撃への対応が更に困難になっている。多くの攻撃者はファイルレス攻撃において、標的マシン上に標準搭載されている正規ツールを利用する。最も利用される正規ツールとして、Windows に標準搭載されているコマンドラインシェルおよびスクリプト言語である PowerShell がある。PowerShell における攻撃スクリプトの実行は、ファイルを生成せずにメインメモリ上で行われる。PowerShell を用いることで、プログラムファイルを用いることなくレジストリや Windows API といった Windows システム内にある機能の大半を使用できる。

コンピュータを対象としたサイバー攻撃によるインシデントが発生した際、原因究明のためにデジタル・フォレンジックを用いて、コンピュータに保存されたデータの保全や解析が行われる。従来のデジタル・フォレンジックでは、主にコンピュータの補助記憶装置を対象として解析が進められる。一方で、ファイルレス攻撃のような補助記憶装置に攻撃に関わるファイルを生成しない攻撃において、従来のデジタル・フォレンジックでは対応できない。そのため、PowerShell を用いたファイルレス攻撃によるインシデントが発生した際の主な調査手法として、ログ解析が用いられる。

PowerShell には、操作に関わるログを記録するシステム（以下、「PowerShell ログシステム」とする）が標準で搭載されている。インシデントが発生した際、原因究明のために PowerShell ログシステムで記録されるログ（以下、「PowerShell ログ」とする）を保全し、解析を行う。一方で、攻撃に関わる痕跡（以下、「アーティファクト」とする）を標的に極力残さないような手法であるアンチ・フォレンジックを攻撃に組み込むことにより、標的マシン上に PowerShell ログが残らない。PowerShell ログシステムに対するアンチ・フォレンジックとして、「PowerShell ログシステムの無効化」、「PowerShell ログの改ざん・削除」および「PowerShell スクリプトの難読化」がある。PowerShell を用いたファイルレス攻撃において、これらのアンチ・フォレンジックを組み込まれると、標準の PowerShell ログシステムでは調査に必要なログを保全できない。そこで、本研究では PowerShell 内部に独自ログシステムを組み込み直接、ローカル外ログサーバにログを保全する手法を提案する。

## 2. ログ保全の従来手法と課題

### 2.1 標準 PowerShell ログシステム

PowerShell を用いたファイルレス攻撃によるインシデントが発生した際、多くの場合 PowerShell ログシステムで記録される PowerShell ログを収集・解析することになる。Windows イベントログの一つとして管理される PowerShell ログは、PowerShell のコマンドラインもしくは Windows 標準搭載の Event Viewer で確認できる。

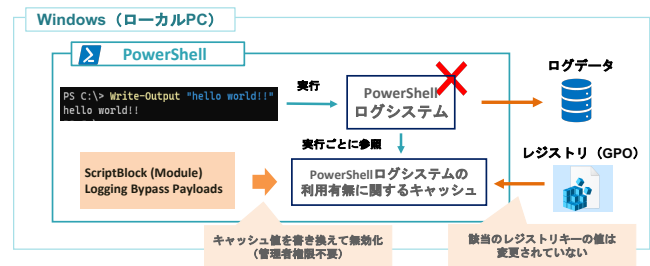


図1 標準の PowerShell ログシステムの無効化手法

システム管理者がレジストリの直接操作、もしくはグループポリシーを操作して PowerShell ログシステムの設定を行うことができる。PowerShell ログシステムの設定は、Windows レジストリのレジストリキー “\HKEY\_LOCAL\_MACHINE\SOFTWARE\Policies\Microsoft\Windows\PowerShell\” 以下にある。PowerShell ログシステムは PowerShell の起動時に上記のレジストリキーの値を System.Management.Automation.Utils の s.cachedPoliciesFromRegistry フィールドにキャッシュする。PowerShell ログシステムの有効・無効は、上記のレジストリキーの直接参照でなく、PowerShell 起動時にキャッシュされた値の参照によって決定する。

Ryan Cobb が発表したが発表した PowerShell ログシステムの無効化手法に「PowerShell ScriptBlock Logging Bypass」がある [1]。図1に示すように、この手法では PowerShell 起動時に該当レジストリキーから読み込まれた PowerShell ログシステムに利用有無に関わるキャッシュの値を書き換えることで、PowerShell ログシステムを無効化する。この手法の強力な点として、攻撃者が PowerShell コンソール上から管理者権限不要でキャッシュ値を書き換えて、無効化できることがある。さらに、この手法はすでに起動された PowerShell 内のキャッシュ値のみを書き換えるため、該当のレジストリキーの値は変更されない。これにより、この手法は Microsoft Defender をはじめとするアンチ・ウイルスソフトウェアに備わるレジストリ変更の監視機能に対しても有効である。

### 2.2 PowerShell ログの改ざん・削除

Windows イベントログの一つとして管理されている PowerShell ログは通常、PowerShell を起動しているローカルマシン内に保存されている。そのため、攻撃者によってローカル内に保存されたログを特定され、改ざん・削除される可能性がある。

### 2.3 関連研究

ログ保全に関する関連研究として、芦野らが提案したセキュリティデバイスとヒステリシス署名を用いてログファイルを保全する研究がある [2]。ヒステリシス署名とは、対象データを署名する際に過去に署名したデータ（署名履歴）を含めて、署名処理を行うものである。これにより、署名されたログファイル間に連鎖構造が築かれるため、攻撃者による一部のログファイルの改ざんを検知できる。ヒステリシス署名は署名の連鎖をたどることでファイルを改変される可能性があるため、芦野らの手法では、セキュリティデバイス内の耐タンパ領域にヒステリシス署名時の署名を保管している。一方で、攻撃者がログシス

テムの無効化に成功した場合、ログ自体が記録されずこの手法を利用できない。

高田ら [3] は、ログファイルのバックアップを複数（任意数）作成し、ローカル内のファイルシステムに保存することでログファイルの改ざんを困難にする手法を提案している。高田らの提案システムは、種々のアプリケーションに組み込むことができるため、攻撃者によるログシステム無効化の影響を受けにくい。本研究の問題点として、ログファイルの安全性が隠蔽箇所数に依存することである。ログを生成するアプリケーションが一般ユーザ権限の場合、書き込み可能なディレクトリが限定され、バックアップの安全性を低下させる可能性がある。そのため、攻撃者により攻撃対象のファイルシステムにある原本のログファイルおよびバックアップのログファイルを発見されやすくなり、改ざんや削除されると復元が困難になる。また、本提案システムは処理を継続して行う必要があるため、コマンド実行のような、短時間で処理されるアプリケーションのログ出力に適用することが困難である。

### 3. 提案手法

#### 3.1 概要

PowerShell を用いたファイルレス攻撃によるインシデントに対し多くの場合、ログ解析が実施される。一方で、攻撃者は PowerShell ログシステムの無効化および PowerShell ログの改ざん・削除というようなアンチ・フォレンジックを攻撃に組み込むことで、ログ解析を妨害する。このような状況に対し、2. 章で述べたような標準の PowerShell ログシステムおよび関連研究の手法を用いても、ログを確実に保全できない。そこで本研究では図 2 に示すように、PowerShell 内部に独自ログシステムを構築し、直接ローカル外ログサーバ（以下、「ログサーバ」とする）にログを保全する手法を提案する。本稿では、この手法を用いたシステム全体を「独自保全システム」と呼ぶこととする。

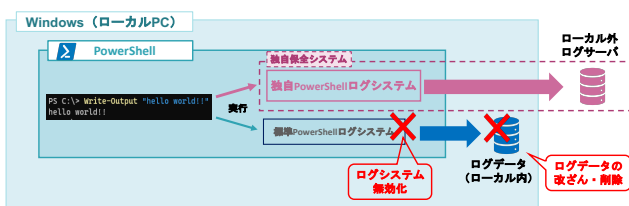


図2 提案手法の概要

#### 3.2 ログ保全のための要件

独自保全システムにおいて、ログを確実に保全するために以下五つの条件を設定した。

- 独自ログシステムは第三者による機能停止を許さず、常時稼働できる。
- ログサーバが受け取るログデータが独自ログシステムから送られたものか判別できる。
- ログ転送路におけるログの改ざんや欠損を検知できる。
- ログサーバに記録されるログの順序が、独自ログシステムで記録されたログの順序と同じである。（順序の入れ替わり

がない）

- 独自ログシステムからログサーバへのログ転送の失敗に対応できる。

#### 3.3 独自ログシステムの組み込み

本手法では、独自ログシステムを PowerShell 実行処理系内にビルドする形で組み込む。つまり、独自ログシステムは PowerShell 実行プロセス内で稼働することになる。そのため、攻撃者を含めたすべての PowerShell ユーザは、独自ログシステムの存在を認知していても、PowerShell 実行処理系に影響を与えず独自ログシステムのみ干渉することが困難である。一方で、PowerShell 内部に独自ログシステムを組み込むため、PowerShell 本体の処理に影響を与える可能性がある。

#### 3.4 ログサーバへの保全

独自ログシステムで記録されたログは、ローカル外のログサーバに送信される。そのため、攻撃者によるローカル内のログに対する改ざん・削除の影響を受けない。一方で、安全で確実なログ保全を実現するためには、何らかの理由による独自ログシステムからログサーバへのログ転送における失敗に対応できる必要がある。そこで、本手法では独自ログシステムおよびログサーバに対し、「署名検証」、「直前ログデータのハッシュ値検証（以下、「ハッシュ値検証」とする）」、「再送機能」を組み込んだ独自保全システムを検討する。

##### 3.4.1 署名検証

ログサーバは、独自ログシステムから送信されたログのみを受け付ける必要がある。また、ログサーバは受け取ったログが第三者によって改ざんされていないことを確認する必要がある。そこで本手法では、独自ログシステムがログサーバに送信するログに署名を付与し、受け取ったログサーバがその署名を検証する。署名およびその検証に用いる鍵は PowerShell 起動時に独自ログシステムおよびログサーバ間で共有する。独自ログシステムは、ログのハッシュ値に対し秘密鍵で暗号化することで署名を生成し、ログデータに付与してログサーバに送信する。ログサーバは受け取った署名を署名生成に用いた秘密鍵に対応する公開鍵で復号し、得られたハッシュ値とログデータ本体のハッシュ値が一致するかを検証を行う。一致すれば署名検証成功となり、独自ログシステムからログサーバに送信されたログが「独自ログシステムから送信されたもの」および「改ざんや欠損がない」ということを証明できる。

##### 3.4.2 ハッシュ値検証

攻撃者はログ解析の妨害目的とし、ログサーバに対し独自ログシステムで記録されたログ以外のデータを送信することが想定できる。本手法ではログデータの署名検証機能を設けているため、ログサーバは独自ログシステム以外から送信されたデータを受け付けない。一方で、攻撃者は独自ログシステムとログサーバの通信路に流れるパケットを盗聴・複製し、そのままログサーバに再送することで非正規なログを差し込むことができる。また、何らかの理由で独自ログシステムで記録されたログの順序とログサーバで記録されるログの順序が入れ替わる可能性も想定できる。

そこで本手法では図 3 のように、直前の送信ログデータの

ハッシュ値を次のログデータに付加し連鎖構造にする。ログサーバは、直前ログデータのハッシュ値と次のログデータに付加されたハッシュ値を検証し、両者が一致すれば正規ログとして受け付ける。このように、ログデータ間の連鎖構造を保つことで、第三者による非正規データの差し込みやログサーバで保存されるログ順序の入れ替わりが無いことを証明できる。

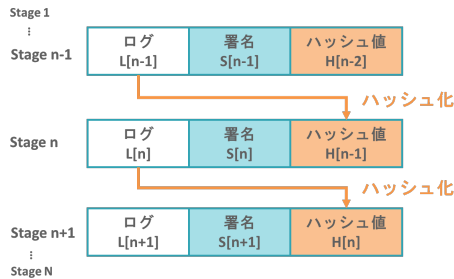


図3 ログデータの連鎖構造

### 3.4.3 再送処理

独自ログシステムからログサーバにログを送信する過程で、何らかの理由によりログ送信に失敗する可能性がある。また、ログサーバにおいて署名検証およびハッシュ値検証に失敗する可能性を想定する必要がある。そこで本手法では、独自ログシステムに対し再送機能を設ける。

## 4. 実装

### 4.1 独自保全システムの動作概要

本提案システムの動作の流れを図4に示す。2023年2月時点のPowerShellの最新版として、Windows PowerShell 5.1およびPowerShell 7.3を利用できる。オープンソースであるPowerShell 7以降では、自身でPowerShellのソースファイルを編集しビルドを行うことで、カスタムしたPowerShellを使用できる。本手法では、独自ログシステムをPowerShell内部処理系内でビルドして組み込むため、オープンソースであるPowerShell 7.3を用いることにした。

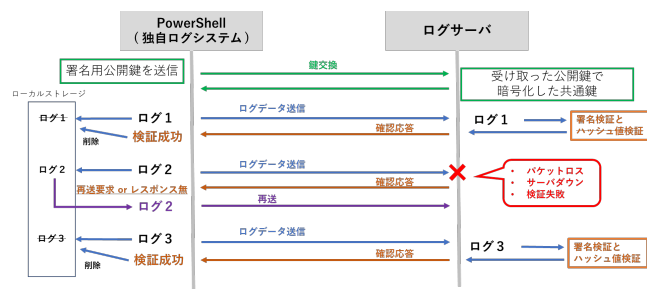


図4 独自ログシステムの動作の流れ

### 4.2 鍵交換

本手法において、署名検証に必要な鍵を独自ログシステムとログサーバで共有しておく必要がある。また、独自ログシステムとログサーバ間の通信路において、第三者（攻撃者）に平文のログデータパケットを盗聴されると、ログ送信の機密性が失われる。本手法では、4.3.1項で述べるUDP通信を用いた

め、独自ログシステム側で送信ログおよび確認応答に対する暗号化機能を組み込む必要がある。そこで、本提案システムは、PowerShellを起動しコンソールセッションを開始する前に図5に示すように、ログの署名および暗号化通信のための鍵ペア（秘密鍵-公開鍵、共通鍵）を交換する。

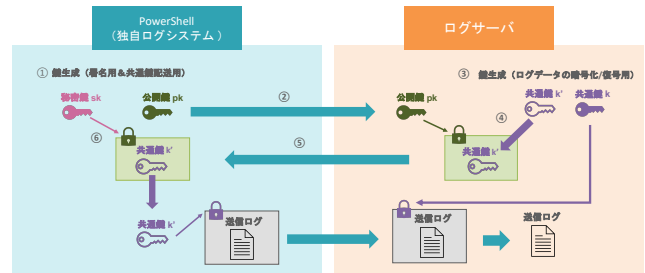


図5 鍵交換の流れ

### 4.3 ログ送信プロトコル

ログサーバにおけるログの署名およびハッシュ値検証を行うために、独自ログシステムとログサーバ間でのログ送信プロトコルを定める必要がある。本システムにおいて、独自ログシステムからログサーバに送信するパケットに図6に示すような情報を付加する。

| ①内の単位はバイト      |                   |                       |             |                             |                |                 |                     |
|----------------|-------------------|-----------------------|-------------|-----------------------------|----------------|-----------------|---------------------|
| ログデータ<br>(可変長) | ログデータの署名<br>(172) | 直前ログデータのハッシュ値<br>(64) | ログ番号<br>(5) | PowerShell<br>プロセス番号<br>(5) | デバイスID<br>(36) | タイムスタンプ<br>(32) | パケット<br>識別番号<br>(4) |

図6 送信ログのフォーマット

#### 4.3.1 UDP通信

独自ログシステムからログサーバにログを確実に送信するためには、信頼性のある通信が保証されるTCPを用いることが望ましい。一方でTCP通信において、独自ログシステムはログを送信した後、ログサーバからの応答を待つ必要がある。ここで何らかの理由でエラーが発生した場合、TCP通信において再送処理が発生し、処理の間はPowerShell本体の処理が中断することになる。TCP通信を用いたログ送信処理とPowerShell本体の処理を別のスレッドで処理すれば、PowerShell本体の処理を中断することなく安全にログを転送できるが、処理が複雑になる。本手法においては、独自ログシステムをPowerShellに組み込むことによるレスポンス低下を避けるため、UDPを用いてログを送信することにした。また、再送処理についてはPowerShell本体処理と別のスレッドで行うことにした。

#### 4.3.2 ログデータの署名

独自ログシステムは、取得したPowerShellログに対する署名を生成する。署名の生成には、.NETクラス・ライブラリに用意されているRSAPKCS1SignatureFormatterクラスのCreateSignatureメソッドを利用した。使用するハッシュアルゴリズムはSHA256で、署名に必要な鍵は4.2項で述べた鍵交換時に生成した秘密鍵を用いる。

#### 4.3.3 直前ログデータのハッシュ値

独自ログシステムでは、N番目のログデータに対し、直前の



独自ログシステムに組み込むことによる PowerShell 本体の実行性能への影響を検証するために、独自ログシステム組み込み有無それぞれの PowerShell において「Write-Output “hello world!!”」コマンドを 50 回実行した際の平均実行時間の差を算出した。

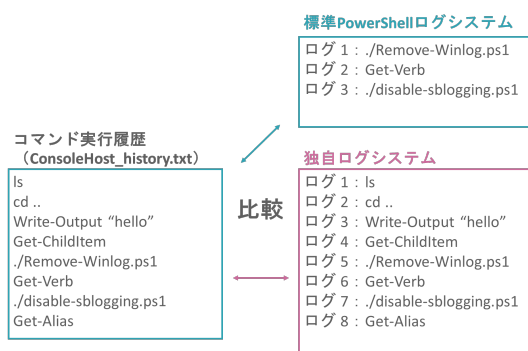


図9 ログ保全可否の評価方法

## 5.3 評価結果

### 5.3.1 評価結果1：ログ保全の可否

被験者10名が使用したPowerShellにおける標準ログシステムおよび独自ログシステムそれぞれのログ保全結果を表1に示す。すべての被験者において、標準ログシステムではログの取りこぼしを確認したが、独自ログシステムではログの取りこぼしがなくログを保全できていた。また、被験者Bと被験者Hにおいて再送処理が発生した。

表1 標準ログシステムと独自ログシステムのログ保全結果

| 被験者      | 標準ログシステム | 独自ログシステム | 実行総数 |
|----------|----------|----------|------|
| A        | × (39)   | ✓        | 1108 |
| B (再送有り) | × (176)  | ✓        | 373  |
| C        | × (230)  | ✓        | 553  |
| D        | × (69)   | ✓        | 209  |
| E        | × (233)  | ✓        | 367  |
| F        | × (54)   | ✓        | 200  |
| G        | × (91)   | ✓        | 214  |
| H (再送有り) | × (29)   | ✓        | 276  |
| I        | × (167)  | ✓        | 386  |
| J        | × (0)    | ✓        | 237  |

✓： ログの取りこぼし無し

×： ログの取りこぼし有り

( ) 内の数は実行総数に対する保全できたログの数を示す。

### 5.3.2 評価結果2：独自ログシステムのオーバーヘッド

表2の通り、独自ログシステム組み込み有無によるPowerShellでの平均実行時間の差は約0.44msであった。

表2 独自ログシステム組み込み有無による実行時間

| 独自ログシステムの組み込み | 平均実行時間    |
|---------------|-----------|
| 無し            | 1.2381 ms |
| 有り            | 1.6793 ms |

## 6. 考 察

本評価において、被験者Bおよび被験者Hで再送処理が発生した。再送処理に失敗した場合、独自ログシステムにおけるログの取りこぼしが発生する。表1に示したように、それぞれの被験者の独自ログシステムにおけるログの取りこぼしがなかった。このことから、再送処理が正確に実行されたといえる。

また、独自ログシステムをPowerShell内部に組み込むことで発生するオーバーヘッドが約0.44msであった。よって、ユーザのPowerShell操作において、独自ログシステムの組み込みによるPowerShell本体の実行性能の低下はほとんど無いと言える。

## 7. 課題と今後の展望

本手法で実装した独自保全システムにおいて、再送処理のためにログを一時的にローカル内に保存している。攻撃者によってこのログを削除される可能性があり、その場合は再送が不可能である。そのため、再送処理用にローカル内に一時保存するログに対し、何らかの保護をする必要がある。

また、本研究で独自ログシステムを組み込んだPowerShellは、オープンソース化されたPowerShell7 (Ver. 6.0以上)であり、これとは別にWindows標準搭載のWindows PowerShell (Ver. 5.1以下)が存在する。現状、両者はシステム上で共存する形となっており、攻撃者は独自ログシステムを組み込んだPowerShellを避けて、片方のPowerShellを使用する可能性が考えられる。

本手法の独自ログシステムをPowerShell内部に組み込むことによる副次的な効果として、PowerShellスクリプトの難読化解除が挙げられる。これはPowerShell内部処理をフックすることで、ユーザの入力段階で難読化されたコマンドやスクリプトを難読化解除した形でロギングできる。本研究において、PowerShellで使用できる(.NET)クラス・ライブラリの難読化解除を実現できることを確認した。(.NET)クラス・ライブラリは攻撃者によるコードインジェクションをはじめとする攻撃で多用されるもので、攻撃者は難読化を施すことでアンチ・ウイルスソフトウェアの検知やログ解析を回避する。今後、フックできるPowerShell内部処理が増えることで、PowerShellスクリプト全体の難読化解除の実現が期待できる。

## 文 献

- [1] Ryan Cobb; PowerShell ScriptBlock Logging Bypass, <https://cobbr.io/ScriptBlock-Logging-Bypass.html> (2017).
- [2] 芦野佑樹, 佐々木良一: セキュリティデバイスとヒステリシス署名を用いたデジタルフォレンジックシステムの提案と評価, 情報処理学会論文誌, Vol. 49, No. 2, pp. 999 - 1009, 2008.
- [3] 高田哲司, 小池英樹: 逃げログ: 削除まで考慮にいたったログ情報保護手法, 情報処理学会論文誌, Vol. 41, No. 3, pp. 823 - 831, 2000.